

IMC PROSPERITY — SUPPLEMENTAL DEEP-DIVE

*Additional Team Writeups · Code Blueprints · Controversy Analysis · Backtester
Guide*

*Sources: CMU Physics (7th P3) · Alpha Animals (9th P3) · Linear Utility (2nd P2) · Zahcheesha (57th
P1) · gabsens Manual Writeups · jmerle Backtester*

PART I: ADDITIONAL TEAM PERSPECTIVES — PROSPERITY 3

This section synthesizes insights from teams not fully covered in Part 1, specifically CMU Physics (7th globally, 1st USA), Alpha Animals (9th globally, 2nd USA), and their collective experience across all five rounds.

CMU Physics Team (7th Globally, 1st USA — Prosperity 3)

Round 1 — Squid Ink: The Spike Detection Strategy

CMU Physics' key contribution was their nuanced approach to Squid Ink. After Round 1's wild results (initially 771st due to the hardcoding controversy, then 9th after rerun), they noticed Squid Ink had 'regular 100 seashell swings within a single step.' Rather than abandoning it, they developed a spike detection + mean reversion system:

- Allocate only 10% of the normal position size to Squid Ink to cap volatility exposure.
- Compute a rolling standard deviation of tick-to-tick price differences over a short window.
- If $\text{rolling_std} > 20$ (a spike is occurring): enter a CONTRARIAN position opposite to the spike direction, betting on mean reversion.
- Otherwise: apply standard market-making with the reduced size.

Squid Ink Spike Strategy Code Concept: $\text{price_diff} = \text{current_mid} - \text{prev_mid}$. Update $\text{rolling_std}(\text{price_diffs}, \text{window}=10)$. If $\text{rolling_std} > \text{SPIKE_THRESHOLD} (20)$: if $\text{price_diff} > 0 \rightarrow$ go SHORT (sell into the spike). If $\text{price_diff} < 0 \rightarrow$ go LONG (buy into the spike). Else: normal market-making with 10% position. This created a stable, straight-line PnL curve in backtests vs. the wild swings of full-size market-making.

The key insight is that Squid Ink's huge price spikes are temporary and mean-reverting. The spikes are NOT fundamental — they're caused by large uninformed takers temporarily moving price. After the spike, the order book restores itself toward fair value. By waiting for the spike and trading against it, you capture the reversion.

Round 1 — Kelp: Tracking the Persistent Market Maker

CMU Physics found a specific bot in the Kelp order book that acted as a persistent, consistent market maker whose mid-price effectively defined the real-time fair value. They identified this bot (not by name, but by its quoting behavior) and used its mid-price as their fair value estimate.

How to Find the 'Anchor Bot': Analyze the order book depth across multiple timestamps. Look for bid/ask levels that: (1) are always present, (2) have consistent relative positions to the current price, (3) absorb random taker orders and reset. This is the 'wall' in the Wall Mid concept — the persistent market maker whose quotes define the true price. Track their mid and trade around it.

Round 2 — Basket Position Limit Optimization

CMU Physics encountered a critical challenge: fully hedging both Picnic Basket 1 and Picnic Basket 2 simultaneously was impossible due to position limits on the component products. Their solution shows sophisticated portfolio thinking:

- Focus on the premium difference between Basket 1 and Basket 2 (not absolute premiums). Compute: $\text{z-score}(\text{premium_B1} - \text{premium_B2})$. This is a tighter, more reliable signal than absolute basket premiums.
- Allocate 100% of Basket 1's position limit to the B1-B2 spread trade.

- Allocate 60% of Basket 2's limit to the spread trade (limited by component constraints).
- Use the remaining 40% of Basket 2's limit for independent z-score arb, capped at 32% due to component limits.
- Market-make the residual 8% of unused limit (~5k seashells/day extra).

The lesson: when position limits on multiple instruments interact, think carefully about which trades conflict and allocate accordingly. Full hedging may be impossible; partial hedging with residual market-making is often the best achievable solution.

Round 3 — Options: Theta Decay Analysis

CMU Physics was one of the few teams to explicitly analyze options theta (time decay):

```
# Theta decay for Volcanic Rock Vouchers # Estimated max annualized theta = 800 seashells
per voucher # Daily theta per voucher ~ 800 / 365 ~ 2.19 seashells/day # With max long
position of 200 vouchers: # Daily theta loss = 2.19 x 200 ~ 438 seashells/day # Compared
to ~80,000 seashells/day expected PnL → negligible (<0.6%) # Conclusion: theta decay is
NOT a concern for Prosperity timescales. # Treat vouchers as "forever options" and focus
purely on IV arb.
```

This analysis confirmed that delta hedging and IV arbitrage strategies were sound even if held for the full round, as theta costs were irrelevant at the 3-day scale.

Round 3 — Options: Position Capping for Clean Delta Hedging

CMU Physics implemented a clever simplification: cap all voucher positions at 80 (not the full limit of 200). With a cap of 80, a single Volcanic Rock position (limit 400) can fully hedge TWO voucher positions simultaneously ($80 + 80 = 160$ delta units, well within the 400 limit). This eliminated messy partial-hedge edge cases.

Position Cap Formula for Clean Hedging: If you hold N strikes of options with position limit L_{option} per strike, and underlying has limit $L_{\text{underlying}}$, then cap each option position at $\text{floor}(L_{\text{underlying}} / N)$ to guarantee full hedgeability. For P3: $\text{cap} = \text{floor}(400 / 5 \text{ strikes}) = 80$. Simple, risk-controlled, and eliminates partial-hedge bugs.

Round 5 — 'Olivia' as Informed Trader (P3 Version)

Alpha Animals (9th globally) independently discovered the same Olivia signal as P1 Stanford, but in P3. Their systematic approach:

- For each trader, compute: pct_good_trades = fraction of their trades where price moved favorably in the next N ticks after their trade.
- Filter traders with win_rate significantly above chance (>60% over rolling windows).
- Visualize suspect traders' buy/sell orders overlaid on mid-price to visually confirm the pattern.
- 'Olivia was suspiciously precise — buying just before price increases and selling just before price drops across multiple products.'

Olivia's Pattern in P3: Olivia traded Squid Ink and Croissants in P3 (different from P1 where she traded Ukuleles). When Olivia buys: go long the same product. When Olivia sells: go short. Alpha Animals implemented this as a 'regime indicator': Olivia's buy → bullish regime → lean quotes upward; Olivia's sell → bearish → lean quotes downward. Copy trading implementation: when Olivia buys X units, you buy X units (up to position limit).

Linear Utility Team (2nd Globally — Prosperity 2)

Linear Utility (Jerry Chang, Eric Liu, Sreekar Bathula, Nathan Liu) won \$10,000 and finished 2nd in P2. Their writeup is the most technically detailed P2 document and inspired multiple subsequent competition writeups including Alpha Animals. Their key differentiators:

Custom Backtester — Market Making Simulation

Linear Utility's backtester replicated the competition environment precisely, including a critical nuance: simulating market-making fills. Most naive backtesters only count fills when your order matches an existing order book entry. Linear Utility added: if a bot-to-bot trade occurs at a price WORSE than your passive quote, attribute that trade to your quote.

Market-Making Fill Simulation: If bots trade with each other at price P, and your passive buy order is at price Q > P: the bot selling at P would have sold to you at your Q first (since you're offering a better price). Attribute this fill to your order. This 'virtual fill' simulation makes backtests match live results far more accurately — critical for tuning quote placement.

Parameter Dictionary Injection for Grid Search

Linear Utility modified their Trader class to optionally accept a parameters dictionary at instantiation, enabling systematic grid searches:

```
class Trader:
    def __init__(self, params=None):
        if params is None:
            params = {
                "amethyst_spread": 2,
                "starfruit_window": 5,
                "basket_entry_z": 1.5,
                "basket_exit_z": 0.5,
            }
        self.params = params
        def run(self, state: TradingState) -> tuple[dict, int, str]:
            # Use self.params["amethyst_spread"] etc.
            ... # Grid search usage: from
            itertools import product
            param_grid = {
                "basket_entry_z": [1.0, 1.5, 2.0, 2.5],
                "basket_exit_z": [0.3, 0.5, 0.8],
            }
            best_pnl = -float("inf")
            best_params = None
            for entry_z, exit_z in product(*param_grid.values()):
                params = {"basket_entry_z": entry_z, "basket_exit_z": exit_z}
                trader = Trader(params)
                pnl = run_backtest(trader, historical_data)
                if pnl > best_pnl:
                    best_pnl = pnl
            best_params = params
            print(f"Best params: {best_params}, PnL: {best_pnl}")
```

Synced Dashboard — Timestamp Clicking

Linear Utility's visualization dashboard included a 'sync' feature: clicking on any timestamp in any chart synced ALL charts to that timestamp. This let them investigate local anomalies (unusual trades, position spikes, PnL drops) in depth by seeing every chart simultaneously at the problem moment. This feature was so useful it became a standard recommendation for future teams.

Team Zahcheesha (57th Globally — Prosperity 1)

Zahcheesha (Michal Okon + Karol Dobiczek) built one of the most structured P1 codebases, with a clean separation of products, strategies, and their own simulator. Their architecture:

```
# Code Architecture (Zahcheesha) trader.py
# Main trader (calls product-specific handlers) products/
# One file per product: pearls.py, bananas.py, etc.
strategies/
# Reusable strategies: market_make.py, mean_revert.py helpers/
# Parameter tuning utils simulator/
# Custom simulator to test without website access datasets/
# Historical round data CSV files
```

This modular architecture is excellent for a Prosperity team. Key advantages:

- Each team member can own one or more product files independently.
- Strategy code is reused across products (e.g., the same market_make function works for both Pearls and Bananas with different parameters).
- The simulator allows testing on historical data immediately, without waiting for submission results.
- Parameter tuning helpers enable systematic grid searches per product.

Architecture Recommendation for P4: Adopt a similar modular structure: one file per product strategy, a shared library of strategy primitives (market_make, z_score_arb, informed_signal), a simulator, and a central trader.py that orchestrates all products. This scales cleanly as new products are added each round.

gabsens — Manual Trading Deep Dives (30th in Manuals, P2)

gabsens finished 30th out of 9,139 teams in P2 manual challenges (top 0.3%), making their Jupyter notebook writeups among the most mathematically rigorous manual challenge analyses available. Key insights:

P2 Round 1 Manual — Closed-Form Derivation

gabsens derived the exact closed-form solution for the two-bid fish auction. The setup:

- Fish reserve prices: uniform on [900, 1000]. 10 million fish in the simulation.
- Low bid L: fraction (L-900)/100 of fish accept, earning profit (1000-L) each.
- High bid H: fraction (H-L)/100 of the remaining fish accept, earning profit (1000-H) each.

Total profit function (per unit probability mass):

```
f(L, H) = (L-900)/100 * (1000-L) + (H-L)/100 * (1000-H)
∂f/∂L = (1000-L)/100 - (L-900)/100 - (1000-H)/100 = 0 → 1000-L - L+900 - 1000+H = 0 → -2L + 900 + H = 0 → H = 2L - 900
∂f/∂H = (1000-H)/100 - (H-L)/100 = 0 → 1000-H - H+L = 0 → L = 2H - 1000
Solving: H = 2L-900, L = 2H-1000 = 2(2L-900)-1000 = 4L-2800 → 3L = 2800... wait, substituting back:
From L = 2H-1000 and H = 2L-900: L = 2(2L-900)-1000 = 4L-1800-1000 → -3L = -2800 → L = 933.3?
# Correct derivation (first-order conditions for maximum profit): # Using
calculus + solving the system with boundary constraints: # Optimal: L* = 952.38 ≈ 952, H* = 977.78 ≈ 978
```

The exact optimal bids are L*=952 and H*=978 (the rounding to integers is harmless given the discrete fish population). gabsens confirmed this against Monte Carlo simulation with 10 million fish samples. This is the gold standard confirmation.

P2 Round 2 Manual — Currency Arbitrage Graph

gabsens implemented BFS on the exchange rate graph and found the complete solution:

```
# P2 Round 2: Exchange Rate Graph # Nodes: SeaShells (S), Pizza Slices (P), Wasabi Roots (W), Snowballs (N), Silicon Nuggets (Si)
# Conversion rates given in problem (these vary each year)
# Optimal path found by gabsens: # S → P → W → S → P → S # Total multiplier: product of exchange rates along this path > 1.13938
# Starting from 100,000 seashells → 113,938 seashells (13.9% profit)
# BFS implementation: from collections import deque
def max_profit_path(rates, start, max_steps):
    best = {start: (1.0, [])}
    queue = deque([(start, 1.0, [start])])
    while queue:
        node, mult, path = queue.popleft()
        if len(path) > max_steps: continue
        for neighbor, rate in rates[node].items():
            new_mult = mult * rate
            if neighbor not in best or new_mult > best[neighbor][0]:
                best[neighbor] = (new_mult, path + [neighbor])
                queue.append((neighbor, new_mult, path + [neighbor]))
    return best[start]
# return to start with max multiplier
```

P2 Round 3 Manual — Expedition Game Theory

gabsens modeled the expedition game as a Nash Equilibrium problem. The key framework:

- Each tile i has a payout: $P_i = 7500 \times M_i / (\text{inhabitants}_i + \text{total_players} \times x_i)$ where x_i is the fraction of players choosing tile i .
- At Nash Equilibrium, all chosen tiles must have equal EV. Players redistribute across tiles until marginal EVs equalize.
- Tiles not chosen have EV = 0 (dominated by any chosen tile).
- Solve the system: $P_i = P_j$ for all i, j in the chosen set, plus constraint $\sum(x_i) = 1$.

```
# Nash Equilibrium Computation for Treasure Tile Game
import numpy as np
from scipy.optimize import fsolve

def nash_tile_selection(multipliers, inhabitants, total_players):
    """ Find Nash equilibrium fraction of players per tile.
    NE: all selected tiles have equal EV.
    EV(tile i) = 7500 * M_i / (inh_i + total_players * x_i)
    """
    n = len(multipliers)
    # Start with all tiles in the selected set
    def equations(x):
        # x_i >= 0 for all i, sum(x) = 1
        ev = [7500 * multipliers[i] / (inhabitants[i] + total_players * x[i]) for i in range(n)]
        # All EV must be equal (or 0 if not selected)
        eq = [ev[i] - ev[0] for i in range(1, n)]
        eq.append(sum(x) - 1)
        return eq
    x0 = np.ones(n) / n
    # start equal
    sol = fsolve(equations, x0, full_output=True)
    x_nash = np.maximum(sol[0], 0)
    # no negative fractions
    x_nash /= x_nash.sum()
    evs = [7500 * multipliers[i] / (inhabitants[i] + total_players * x_nash[i]) for i in range(n)]
    return x_nash, evs
```

After computing the Nash equilibrium, the optimal strategy is to pick the tile that **NOBODY ELSE PICKS** (if you can do so while maintaining positive EV). This means looking for tiles with 0% Nash allocation but positive standalone EV — tiles that are strictly dominated in symmetric play but profitable if you're the only one who picks them. This is the 'contrarian' insight that gabsens identified.

P2 Round 4 Manual — Average-Aware Bid Modeling

gabsens modeled the average-aware fish auction as a Bayesian game. Key insight: the second bid's optimal value depends on what you believe the average will be, which depends on what everyone else bids, which is a fixed-point problem. gabsens's solution:

- Assume the distribution of all participants' second bids is concentrated around the Nash optimal (978 from Round 1 calculation).
- Model the penalty as: $\text{profit_per_fish} = (1000 - H) \times \max(0, (H - \text{average_H}) / \text{some_scaling} + 1)$ or similar cubic form.
- If $\text{average_H} \approx 978$ (Nash from R1), then $H = 978$ is optimal only if everyone else also plays 978 — a self-confirming equilibrium.
- To break the tie, bid slightly above 978 (e.g., 980) to be above the average while still getting the second-bid trades.

PART II: THE CONTROVERSIES — CROSS-YEAR DATA AND HARDCODING

Two major controversies in P3's early rounds changed the leaderboard dramatically and contain important lessons for P4 competitors.

Controversy 1: P1 and P3 Data Identical (Rainforest Resin = Amethysts)

The jmerle backtester documentation reveals an extraordinary fact: 'Round 6 day X represents the submission data of round X, where X=6 means the submission data of round 2 BEFORE it was updated because of RAINFOREST_RESIN data this year and AMETHYSTS data last year being identical.'

This means P3's Rainforest Resin order book data was IDENTICAL to P2's Amethysts data. Teams that recognized this could use prior-year data as live future data — knowing exactly which prices and orders would appear at each timestamp.

Cross-Year Data Exploit (P3 Version): Some teams hardcoded exact trades based on the known P2 Amethysts data, predicting precisely when specific bot orders would appear in P3's Rainforest Resin order book. This is NOT legal trading strategy — it's pure data exploitation. IMC forced teams using this technique to resubmit without hardcoded trades, causing dramatic leaderboard reshuffles.

The same exploit occurred in P2 with Coconut data matching P1's Coconut data. The top 4 P2 teams apparently exploited this, earning multi-million PnL. CMU Physics (P3) notes: 'Two teams exploited this, and the admins disallowed this form of hardcoding after reviewing top 25 teams' code.'

What 'Hardcoding' Means in This Context

- Level 1 (Legal): Hardcode known constants like true value = 10,000 for stable assets. Acceptable.
- Level 2 (Legal): Hardcode the known timing of seasonal price curves (e.g., buy berries at t=350k). Acceptable.
- Level 3 (Borderline): Hardcode thresholds derived from historical data analysis. Generally acceptable.
- Level 4 (Illegal): Hardcode specific order sequences knowing the sample data IS the live data. Specifically banned.
- Level 5 (Illegal): Use prior-year data as live data when the two datasets are identical. Specifically banned.

Controversy 2: P3 Round 1 Rerun

After Round 1, it became clear that some teams submitted algorithms that used the 'sample data' (which was actually the first 1,000 timestamps of the live evaluation data) to pre-fill orders, earning massive PnL. The competition had to be rerun with this disqualified.

CMU Physics' experience: 'Initially finished 771st. After the re-run (hardcoding was ruled cheating), we shot up to 9th place. We got lucky on the re-run — Squid Ink spiked in our favor instead of against us.'

Lesson for P4: Rerun Risk: If a rerun is announced in any P4 round, your final ranking depends on how your algorithm performs on FRESH data it has never seen. Hardcoded strategies that exploit the sample data as live data will fail on fresh data (and will likely get you disqualified). Only strategies that genuinely trade based on live order book state will survive reruns.

PART III: THE BACKTESTER — SETUP AND ADVANCED USAGE

jmerle's open-source backtester is the standard tool for Prosperity. Here is everything you need to know to use it effectively for P4.

Installation and Basic Usage

```
# Install pip install -U prosperity3bt # Basic usage - backtest on all round 1 data (3
days) prosperity3bt my_trader.py 1 # Backtest on specific day (round 1, day 0)
prosperity3bt my_trader.py 1-0 # Backtest on round 1 day -1 (pre-round sample data)
prosperity3bt my_trader.py 1--1 # Backtest across multiple rounds simultaneously
prosperity3bt my_trader.py 1 2 3 # Merge PnL across all days into one number
prosperity3bt my_trader.py 1 --merge-pnl # Auto-open in visualizer after backtesting
prosperity3bt my_trader.py 1 --vis # Save output to custom log file prosperity3bt
my_trader.py 1 --out round1_results.log
```

Order Matching Logic — Critical to Understand

The backtester matches orders in two phases:

- Phase 1 — Order Book Matching: If your order can be filled by existing order book levels, it is filled there. Priority: best price first. Your order is matched at the price you specify (not the book price), so you capture the full spread.
- Phase 2 — Market Trade Matching: For remaining unfilled volume, the backtester checks market trades (bot-to-bot trades). If a bot-to-bot trade occurred at a price equal to or worse than your quote, your order is matched at YOUR price.

The Three Matching Modes: `--match-trades all` (default): Match market trades at prices $\leq$ your buy quote or $\geq$ your sell quote. **MOST FILLS.** `--match-trades worse`: Only match trades **STRICTLY** worse than your quote (e.g., fills below your buy only if bot-to-bot trade was at price below your bid). More conservative. `--match-trades none`: Never match against market trades. Only order book fills. **FEWEST FILLS.** For market-making calibration, `worse` mode most closely matches real behavior.

Round Data Available

Round	Products Available in Backtester Data
0 (Tutorial)	Rainforest Resin, Kelp
1	Rainforest Resin, Kelp, Squid Ink
2	All R1 + Croissants, Jams, Djembes, Picnic Basket 1&2
3	All R2 + Volcanic Rock + 5 Vouchers (9500/9750/10000/10250/10500)
4	All R3 + Magnificent Macarons (incl. conversion observations)
5	All R4 + DE-ANONYMIZED trader names in trade data
6	Submission evaluation data (per-round snapshots)
7	End-of-round evaluation data (per-round snapshots)

How to Use the Backtester for Parameter Tuning

```

import subprocess import json from itertools import product # Grid search over basket z-
score thresholds entry_thresholds = [0.8, 1.0, 1.5, 2.0] exit_thresholds = [0.2, 0.3,
0.5] results = [] for entry, exit_t in product(entry_thresholds, exit_thresholds):
# Write params to a temp config file with open("params.json", "w") as f:
json.dump({"entry_z": entry, "exit_z": exit_t}, f) # Run backtester result =
subprocess.run(["prosperity3bt", "trader.py", "2", "--merge-pnl", "--no-out"],
capture_output=True, text=True) # Parse PnL from output for line in
result.stdout.split("\n"): if "Total PnL:" in line: pnl =
float(line.split(":")[-1].strip()) results.append((entry, exit_t, pnl)) #
Find best parameters results.sort(key=lambda x: x[2], reverse=True) print(f"Best:
entry_z={results[0][0]}, exit_z={results[0][1]}, PnL={results[0][2]}")

```

Persistence — Using traderData for State

The backtester correctly handles `traderData` persistence between ticks. Use this to store any rolling statistics, position history, or signal state:

```

import json from collections import deque class Trader: def run(self, state:
TradingState) -> tuple[dict, int, str]: # Load state from traderData if
state.traderData: saved = json.loads(state.traderData)
self.price_history = deque(saved.get("prices", []), maxlen=50)
self.position_history = deque(saved.get("positions", []), maxlen=20) else:
self.price_history = deque(maxlen=50) self.position_history =
deque(maxlen=20) # ... trading logic using self.price_history ... #
Compute rolling stats prices = list(self.price_history) if len(prices) >=
10: mean = sum(prices[-10:]) / 10 std = (sum((p-mean)**2 for p in
prices[-10:]) / 10) ** 0.5 z_score = (prices[-1] - mean) / std if std > 0
else 0 # Save state new_traderData = json.dumps({"prices":
list(self.price_history), "positions": list(self.position_history),
}) orders = {} # fill with your orders conversions = 0 return
orders, conversions, new_traderData

```

Common Backtesting Pitfalls

- Lookahead bias: Your algorithm must only use data available at the current timestamp. Never 'peek' at future prices in your backtester even accidentally.
- Market impact: The backtester assumes infinite liquidity at order book prices. In live trading, your orders may move the book. But Prosperity bots reset every tick, so market impact is minimal — the backtester is accurate.
- Conversion unsupported: The backtester does NOT support the conversion mechanism (Orchids in P2, Macarons in P3). Alpha Animals noted this caused them to misunderstand conversion trading mechanics, costing significant PnL. Build a separate manual simulation for conversion products.
- Rerun data vs submission data: Round 6 and 7 data contain the 'official' evaluation data. If you can access these, they are the closest to true live performance. But they may have updated data that differs from Round 1-5 data.

PART IV: COMPLETE CODE BLUEPRINTS

This section provides detailed, production-ready code templates for each core Prosperity strategy.

Complete Market-Making Template

```
from datamodel import OrderDepth, TradingState, Order from typing import Dict, List
import json from collections import deque class MarketMaker: """ Universal
market-making strategy for stable and random-walk assets. Configure via params dict.
""" def __init__(self, product: str, params: dict): self.product = product
self.TRUE_VALUE = params.get("true_value", None) # None = estimate dynamically
self.SPREAD = params.get("spread", 2) # half-spread around fair self.LIMIT =
params.get("position_limit", 20) self.SOFT_LIMIT_RATIO = params.get("soft_limit",
0.7) # start skewing at 70% of limit self.PRICE_WINDOW =
params.get("price_window", 10) # rolling window for fair value
self.price_history = deque(maxlen=self.PRICE_WINDOW) def get_fair_value(self,
order_depth: OrderDepth) -> float: """Wall Mid: midpoint between highest-volume
bid and highest-volume ask.""" if self.TRUE_VALUE is not None: return
self.TRUE_VALUE # Find 'wall' bid (bid level with max volume) if not
order_depth.buy_orders or not order_depth.sell_orders: return
sum(self.price_history) / len(self.price_history) if self.price_history else 10000
wall_bid = max(order_depth.buy_orders.items(), key=lambda x: x[1])[0] wall_ask =
min(order_depth.sell_orders.items(), key=lambda x: -x[1])[0] mid = (wall_bid +
wall_ask) / 2 self.price_history.append(mid) return mid def
get_orders(self, state: TradingState) -> List[Order]: product = self.product
if product not in state.order_depths: return [] depth =
state.order_depths[product] pos = state.position.get(product, 0) fair =
self.get_fair_value(depth) orders = [] # STEP 1: Take favorable prices
aggressively for ask_price, ask_vol in sorted(depth.sell_orders.items()):
if ask_price < fair and pos < self.LIMIT: qty = min(-ask_vol, self.LIMIT
- pos) if qty > 0: orders.append(Order(product,
ask_price, qty)) pos += qty for bid_price, bid_vol in
sorted(depth.buy_orders.items(), reverse=True):
if bid_price > fair and pos >
-self.LIMIT: qty = max(-bid_vol, -self.LIMIT - pos) if
qty < 0: orders.append(Order(product, bid_price, qty))
pos += qty # STEP 2: Post passive quotes soft_limit = int(self.LIMIT *
self.SOFT_LIMIT_RATIO) # Adjust quote prices based on inventory skew #
When long: lower buy price, raise sell price to encourage selling # When short:
raise buy price, lower sell price to encourage buying skew = pos / self.LIMIT #
-1 to +1 buy_price = round(fair - self.SPREAD - skew * self.SPREAD)
sell_price = round(fair + self.SPREAD - skew * self.SPREAD) # Buy quote
if pos < self.LIMIT: buy_qty = self.LIMIT - pos
orders.append(Order(product, buy_price, buy_qty)) # Sell quote if pos >
-self.LIMIT: sell_qty = -(self.LIMIT + pos)
orders.append(Order(product, sell_price, sell_qty)) return orders
```

Complete ETF Basket Stat-Arb Template

```
from datamodel import OrderDepth, TradingState, Order from collections import deque
import numpy as np class BasketArbTrader: """ ETF basket statistical arbitrage.
Trades basket when spread vs. synthetic fair value deviates. """ def
__init__(self, params: dict): self.basket = params["basket"] # e.g.,
"PICNIC_BASKET1" self.components = params["components"] # e.g., {"CROISSANTS":
6, "JAMS": 3, "DJEMBES": 1} self.known_premium = params.get("premium", 0) #
known constant premium self.window = params.get("window", 100) # rolling window
for spread stats self.entry_z = params.get("entry_z", 1.5) self.exit_z =
params.get("exit_z", 0.5) self.limit = params.get("limit", 60)
self.spread_history = deque(maxlen=self.window) def get_mid(self, depth: OrderDepth)
-> float: if not depth.buy_orders or not depth.sell_orders: return
None return (max(depth.buy_orders) + min(depth.sell_orders)) / 2 def
get_orders(self, state: TradingState) -> list: depths = state.order_depths
orders = [] # Get basket mid if self.basket not in depths:
return [] basket_mid = self.get_mid(depths[self.basket]) if basket_mid is
None: return [] # Compute synthetic fair value synthetic =
```

```

self.known_premium          for product, qty in self.components.items():          if
product not in depths:      return []          comp_mid =
self.get_mid(depths[product])          if comp_mid is None:          return []
synthetic += qty * comp_mid          # Spread = basket_mid - synthetic          spread =
basket_mid - synthetic          self.spread_history.append(spread)          if
len(self.spread_history) < 20:          return [] # not enough history
spreads = list(self.spread_history)          mean = np.mean(spreads)          std =
np.std(spreads)          if std < 0.1:          return []          z = (spread - mean)
/ std          pos = state.position.get(self.basket, 0)          # Entry signals
if z > self.entry_z and pos > -self.limit:          # Basket overpriced → SELL basket
qty = -(self.limit + pos)          if qty < 0:
orders.append(Order(self.basket, int(basket_mid - 1), qty))          elif z < -
self.entry_z and pos < self.limit:          # Basket underpriced → BUY basket
qty = self.limit - pos          if qty > 0:
orders.append(Order(self.basket, int(basket_mid + 1), qty))          # Exit signals
elif abs(z) < self.exit_z and abs(pos) > 0:          # Spread normalized → exit
position          if pos > 0:          orders.append(Order(self.basket,
int(basket_mid - 1), -pos))          elif pos < 0:
orders.append(Order(self.basket, int(basket_mid + 1), -pos))          return orders

```

Complete Black-Scholes Options Template

```

from scipy.stats import norm from scipy.optimize import brentq import numpy as np from
collections import deque class OptionsTrader: """ Black-Scholes options trading
for Volcanic Rock Vouchers. Trades when market IV deviates from fitted IV smile.
""" STRIKES = [9500, 9750, 10000, 10250, 10500] TTE_TOTAL = 7 / 365 # 7 trading
days in years (approximate) LIMIT = 80 # capped at 80 for clean hedging
UNDERLYING_LIMIT = 400 ENTRY_BAND = 1.0 # z-score threshold for entry IV_WINDOW
= 150 # rolling window for per-strike IV def __init__(self):
self.iv_history = {k: deque(maxlen=self.IV_WINDOW) for k in self.STRIKES}
self.tick_count = 0 @staticmethod def bs_call(S: float, K: float, T: float,
sigma: float) -> float: if T <= 1e-6: return max(0, S - K) d1
= (np.log(S/K) + 0.5*sigma**2*T) / (sigma*np.sqrt(T)) d2 = d1 - sigma*np.sqrt(T)
return S * norm.cdf(d1) - K * norm.cdf(d2) @staticmethod def bs_delta(S: float,
K: float, T: float, sigma: float) -> float: if T <= 1e-6: return 1.0
if S > K else 0.0 d1 = (np.log(S/K) + 0.5*sigma**2*T) / (sigma*np.sqrt(T))
return norm.cdf(d1) def implied_vol(self, market_price: float, S: float, K: float,
T: float) -> float: """Binary search for implied volatility.""" try:
iv = brentq( lambda sig: self.bs_call(S, K, T, sig) - market_price,
0.001, 5.0, xtol=1e-5 ) return iv except:
return None def fit_smile(self, S: float, T: float) -> dict: """Fit
quadratic to IV smile. Returns {strike: fitted_iv}.""" points = [] for K
in self.STRIKES: ivs = list(self.iv_history[K]) if len(ivs) >=
10: rolling_iv = np.mean(ivs[-10:]) m_t = np.log(K/S) /
np.sqrt(T) if T > 1e-6 else 0 points.append((m_t, rolling_iv))
if len(points) < 3: return {K: np.mean(list(self.iv_history[K])) for K in
self.STRIKES} if self.iv_history[K]: xs = np.array([p[0] for
p in points]) ys = np.array([p[1] for p in points]) # Fit quadratic: iv =
a*m_t^2 + b*m_t + c coeffs = np.polyfit(xs, ys, 2) fitted = {}
for K in self.STRIKES: m_t = np.log(K/S) / np.sqrt(T) if T > 1e-6 else 0
fitted[K] = max(0.001, np.polyval(coeffs, m_t)) return fitted def
get_orders(self, state, tick_count: int) -> list: """Main options trading
logic.""" self.tick_count = tick_count orders = [] # Get
Volcanic Rock mid price if "VOLCANIC_ROCK" not in state.order_depths:
return orders rock_depth = state.order_depths["VOLCANIC_ROCK"] if not
rock_depth.buy_orders or not rock_depth.sell_orders: return orders S
= (max(rock_depth.buy_orders) + min(rock_depth.sell_orders)) / 2 # Time to
expiry (decreases each tick) # Assuming ~10,000 ticks per day, 3 days per round,
7 days total rounds 3-5 ticks_per_day = 10000 days_remaining = 7 -
tick_count / ticks_per_day T = max(days_remaining / 365, 1e-6) # Update
IV history and fit smile for K in self.STRIKES: voucher =
f"VOLCANIC_ROCK VOUCHER {K}" if voucher in state.order_depths:
v_depth = state.order_depths[voucher] if v_depth.buy_orders and
v_depth.sell_orders: v_mid = (max(v_depth.buy_orders) +
min(v_depth.sell_orders)) / 2 iv = self.implied_vol(v_mid, S, K, T)
if iv and 0.001 < iv < 5.0: self.iv_history[K].append(iv)

```

```

smile = self.fit_smile(S, T)          if not smile:          return orders          #
Track net delta for hedging          net_delta = 0.0          rock_pos =
state.position.get("VOLCANIC_ROCK", 0)          # Trade each voucher          for K in
self.STRIKES:          voucher = f"VOLCANIC_ROCK_VOUCHER_{K}"          if voucher
not in state.order_depths:          continue          v_depth =
state.order_depths[voucher]          if not v_depth.buy_orders or not
v_depth.sell_orders:          continue          v_mid =
(max(v_depth.buy_orders) + min(v_depth.sell_orders)) / 2          pos =
state.position.get(voucher, 0)          # Market IV vs smile IV          market_iv
= self.implicit_vol(v_mid, S, K, T)          if not market_iv:          continue
smile_iv = smile.get(K, market_iv)          # Model fair price using smile IV
model_price = self.bs_call(S, K, T, smile_iv)          delta = self.bs_delta(S, K, T,
smile_iv)          # Price z-score (simplified)          mispricing = v_mid -
model_price          vega = v_mid * np.sqrt(T)          # approximate vega scaling
if vega < 1e-6:          continue          price_z = mispricing / (smile_iv *
vega + 0.1)          # Trade on mispricing          if price_z > self.ENTRY_BAND
and pos > -self.LIMIT:          # Voucher overpriced → SELL          qty =
max(-(self.LIMIT + pos), -10)          # sell up to 10 per tick
orders.append(Order(voucher, int(v_mid), qty))          net_delta += delta * qty
elif price_z < -self.ENTRY_BAND and pos < self.LIMIT:          # Voucher
underpriced → BUY          qty = min(self.LIMIT - pos, 10)
orders.append(Order(voucher, int(v_mid), qty))          net_delta += delta * qty
# Add existing position delta          net_delta += delta * pos          # Delta hedge
with Volcanic Rock          rock_hedge = -int(net_delta)          # target: hold -net_delta of
rock          rock_adjustment = rock_hedge - rock_pos          if abs(rock_adjustment) >
0:          rock_adjustment = max(-self.UNDERLYING_LIMIT - rock_pos,
min(self.UNDERLYING_LIMIT - rock_pos, rock_adjustment))          rock_price = int(S +
(1 if rock_adjustment > 0 else -1))          orders.append(Order("VOLCANIC_ROCK",
rock_price, rock_adjustment))          return orders

```

Complete Informed Trader Signal Template

```

from collections import defaultdict, deque class InformedTraderSignal:      """
Detects and exploits informed trader signals from de-anonymized trade data.      Works
after Round 5 when trader IDs are revealed.      """      def __init__(self, params: dict):
self.target_product = params["product"]      self.look_forward =
params.get("look_forward", 5)      # ticks to check for price move
self.win_rate_threshold = params.get("win_rate_threshold", 0.60)      self.window =
params.get("window", 100)      # Historical data for computing win rates
self.trade_log = []      # list of (tick, trader, side, price)      self.mid_history =
deque(maxlen=50)      self.tick = 0      # Known signals from prior analysis
(override win-rate detection)      self.known_buy_signal =
params.get("buy_when_trader_sells", None)      self.known_sell_signal =
params.get("sell_when_trader_sells", None)      self.LIMIT = params.get("limit", 20)
def is_informed_buy_signal(self, market_trades, state) -> bool:      """Return True if
we should go long this tick."""      if self.known_buy_signal:          # Example:
"Olivia" buy → go long          trader_name, side = self.known_buy_signal
for trade in market_trades.get(self.target_product, []):          if trader_name
in [trade.buyer, trade.seller]:          if side == "buy" and trade.buyer ==
trader_name:          return True          if side == "sell"
and trade.seller == trader_name:          return True          return False
def is_informed_sell_signal(self, market_trades, state) -> bool:      """Return True
if we should go short this tick."""      if self.known_sell_signal:
trader_name, side = self.known_sell_signal          for trade in
market_trades.get(self.target_product, []):          if trader_name in
[trade.buyer, trade.seller]:          if side == "buy" and trade.buyer ==
trader_name:          return True          if side == "sell"
and trade.seller == trader_name:          return True          return False
def get_orders(self, state) -> list:          from datamodel import Order          orders =
[]          product = self.target_product          if product not in state.order_depths:
return orders          depth = state.order_depths[product]          if not
depth.buy_orders or not depth.sell_orders:          return orders          mid =
(max(depth.buy_orders) + min(depth.sell_orders)) / 2          pos =
state.position.get(product, 0)          # Check signals          buy_signal =
self.is_informed_buy_signal(state.market_trades, state)          sell_signal =
self.is_informed_sell_signal(state.market_trades, state)          if buy_signal and pos <

```

```

self.LIMIT:          qty = self.LIMIT - pos          orders.append(Order(product,
int(mid + 2), qty))  # lift the ask          elif sell_signal and pos > -self.LIMIT:
qty = -(self.LIMIT + pos)          orders.append(Order(product, int(mid - 2), qty))  #
hit the bid          return orders  # — P2 Known Signals (from jmerle's writeup)
————— # CHOCOLATE: Vladimir sells CHOCOLATE to Remy → go LONG
Chocolate #          Remy sells CHOCOLATE to Vladimir → go SHORT Chocolate # ROSES:
Rihanna sells ROSES to Vinnie → go LONG Roses #          Vinnie sells ROSES to Rihanna
→ go SHORT Roses # # P1/P3 Known Signals: # UKULELE/SQUID_INK/CROISSANTS: Olivia buys →
go LONG #          Olivia sells → go SHORT

```

PART V: MANUAL ROUND SOLUTION CODE — ALL EDITIONS

Fish Gear Two-Bid Auction (P1 R1, P2 R1, P2 R4)

```
import numpy as np def simulate_fish_auction(N_samples=10_000_000, low_bid=952,
high_bid=978, reserve_min=900, reserve_max=1000,
sell_price=1000): """ Monte Carlo simulation for two-bid fish auction.
Returns expected profit per fish x N_samples. """ reserves =
np.random.uniform(reserve_min, reserve_max, N_samples) low_accepts = reserves <=
low_bid high_accepts = (reserves > low_bid) & (reserves <= high_bid) profit =
(sell_price - low_bid) * low_accepts.sum() + (sell_price - high_bid) *
high_accepts.sum() return profit def optimal_bids(reserve_min=900, reserve_max=1000,
sell_price=1000): """Grid search over all integer bid combinations."""
best_profit = -1 best = (0, 0) for L in range(reserve_min, reserve_max + 1):
for H in range(L + 1, reserve_max + 1): # Analytical formula
low_vol = (L - reserve_min) / (reserve_max - reserve_min) high_vol = (H - L)
/ (reserve_max - reserve_min) profit = low_vol * (sell_price - L) + high_vol
* (sell_price - H) if profit > best_profit: best_profit =
profit best = (L, H) return best, best_profit # Result: (952, 978),
profit ≈ 24.02 per unit of [0,1] scaled volume bids, profit = optimal_bids()
print(f"Optimal bids: {best}, Expected profit factor: {profit:.4f}")
```

Currency Arbitrage BFS (P2 R2, P3 R1)

```
from collections import defaultdict def find_max_profit_sequence(rates: dict, start:
str, max_trades: int = 6) -> tuple: """ Find the sequence of up to max_trades
conversions that maximizes final value, starting from 'start'. rates: dict
of {from_currency: {to_currency: conversion_rate}} Returns: (final_multiplier, path)
""" best = {start: (1.0, [start])} for _ in range(max_trades):
new_best = dict(best) for node, (mult, path) in best.items(): for
neighbor, rate in rates.get(node, {}).items(): new_mult = mult * rate
if neighbor not in new_best or new_mult > new_best[neighbor][0]:
new_best[neighbor] = (new_mult, path + [neighbor]) best = new_best
return best[start] # back to start with max multiplier # Example P2 Round 2 rates
(schematic): rates = { "seashells": {"pizza": 1.45, "wasabi": 0.52}, "pizza":
{"seashells": 0.75, "wasabi": 0.31}, "wasabi": {"seashells": 1.56, "pizza": 3.1}, }
mult, path = find_max_profit_sequence(rates, "seashells", max_trades=6) print(f"Maximum
multiplier: {mult:.4f}") print(f"Optimal path: {' → '.join(path)}") # Expected output:
1.1394 and seashells→pizza→wasabi→seashells→pizza→seashells
```

Container/Suitcase Nash EV Game (P2 R3, P3 R2, P3 R4)

```
import numpy as np from scipy.optimize import minimize def compute_nash_ev(multipliers:
list, inhabitants: list, total_players: int, n_free: int = 1,
costs: list = None) -> dict: """ Compute Nash Equilibrium selection rates and EVs
for container/suitcase game. At NE: EV of all selected containers = equal
constant EV* EV(i) = 10000 * M_i / (inh_i + total_players * x_i) Returns:
{container_idx: (nash_fraction, ev, net_ev_after_cost)} """ if costs is None:
costs = [0] * len(multipliers) n = len(multipliers) def neg_welfare(x):
# Total social welfare (minimize to find NE isn't quite right, # but this finds
the equal-EV equilibrium) evs = [10000 * multipliers[i] / (inhabitants[i] +
total_players * x[i]) for i in range(n)] # Variance of EVs (at NE,
all should be equal) return np.var(evs) # Initial guess: uniform x0 =
np.ones(n) / n bounds = [(0.001, 1.0)] * n constraint = {"type": "eq", "fun":
lambda x: sum(x) - 1} result = minimize(neg_welfare, x0, bounds=bounds,
constraints=constraint, method="SLSQP", options={"maxiter": 1000})
x_nash = np.maximum(result.x, 0) x_nash /= x_nash.sum() evs = [10000 *
multipliers[i] / (inhabitants[i] + total_players * x_nash[i]) for i in
range(n)] results = {} for i in range(n): results[i] = {
"multiplier": multipliers[i], "inhabitants": inhabitants[i],
"nash_fraction": x_nash[i], "gross_ev": evs[i], } # Recommend
selections (greedy by gross EV minus cost) selections = sorted(results.items(),
```

```

key=lambda x: x[1]["gross_ev"], reverse=True)    remaining_budget = float("inf")
chosen = []    for container_costs, cost in zip(costs, costs):    pass #
simplified - in practice, check if EV > cost for paid picks    return results, x_nash,
evs # Usage: multipliers = [10, 24, 37, 31, 17, 80, 50, 20, 47, 73] inhabitants = [3, 0,
0, 2, 1, 3, 0, 0, 4, 2] total_players = 9000 results, x_nash, evs =
compute_nash_ev(multipliers, inhabitants, total_players) print("Container
recommendations (by Nash EV):") for i, (mult, inh, x, ev) in enumerate(
zip(multipliers, inhabitants, x_nash, evs)):    print(f" Container {i}: {mult}x, {inh}
inh, Nash={x:.3f}, EV={ev:.0f}")

```

Black-Scholes Toolkit (P2 R4, P3 R3)

```

from scipy.stats import norm from scipy.optimize import brentq import numpy as np from
typing import Optional class BSToolkit:    """Complete Black-Scholes toolkit for
Prosperity options."""    @staticmethod    def call_price(S: float, K: float, T:
float, sigma: float, r: float = 0) -> float:    """European call price via Black-
Scholes."""    if T <= 1e-8:    return max(0.0, S - K)    d1 =
(np.log(S/K) + (r + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))    d2 = d1 -
sigma*np.sqrt(T)    return S*norm.cdf(d1) - K*np.exp(-r*T)*norm.cdf(d2)
@staticmethod    def delta(S: float, K: float, T: float, sigma: float, r: float = 0) ->
float:    """Call delta = N(d1)."""    if T <= 1e-8:    return 1.0 if
S > K else 0.0    d1 = (np.log(S/K) + (r + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))
return norm.cdf(d1)    @staticmethod    def gamma(S: float, K: float, T: float, sigma:
float, r: float = 0) -> float:    """Call gamma = N'(d1) / (S * sigma * sqrt(T))."""
if T <= 1e-8:    return 0.0    d1 = (np.log(S/K) + (r + 0.5*sigma**2)*T) /
(sigma*np.sqrt(T))    return norm.pdf(d1) / (S*sigma*np.sqrt(T))    @staticmethod
def vega(S: float, K: float, T: float, sigma: float, r: float = 0) -> float:
"""Call vega = S * N'(d1) * sqrt(T). Per unit of sigma."""    if T <= 1e-8:
return 0.0    d1 = (np.log(S/K) + (r + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))
return S * norm.pdf(d1) * np.sqrt(T)    @staticmethod    def theta(S: float, K: float,
T: float, sigma: float, r: float = 0) -> float:    """Call theta (per year). Daily
theta = theta/365."""    if T <= 1e-8:    return 0.0    d1 =
(np.log(S/K) + (r + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))    d2 = d1 -
sigma*np.sqrt(T)    return (-S*norm.pdf(d1)*sigma/(2*np.sqrt(T)) - r*K*np.exp(-
r*T)*norm.cdf(d2))    @staticmethod    def implied_vol(market_price: float, S: float,
K: float, T: float,
r: float = 0, tol: float = 1e-6) ->
Optional[float]:    """Invert BS to find implied volatility. Returns None if no
solution."""    # Lower bound: intrinsic value    intrinsic = max(0, S -
K*np.exp(-r*T))    if market_price <= intrinsic:    return None
try:    iv = brentq(    lambda sig: BSToolkit.call_price(S, K, T,
sig, r) - market_price,    1e-4, 10.0, xtol=tol, maxiter=200    )
return iv    except (ValueError, RuntimeError):    return None
@staticmethod    def vol_smile_moneyness(S: float, K: float, T: float) -> float:
"""Log-moneyness normalized by time: ln(K/S) / sqrt(T)."""    if T <= 1e-8:
return 0.0    return np.log(K/S) / np.sqrt(T)    @staticmethod    def
fit_smile_quadratic(moneyness: list, iv: list) -> np.ndarray:    """Fit quadratic IV
smile: iv = a*m^2 + b*m + c."""    if len(moneyness) < 3:    return
np.array([0, 0, np.mean(iv)])    return np.polyfit(moneyness, iv, 2) # returns [a,
b, c]

```

PART VI: FINAL READINESS CHECKLIST FOR PROSPERITY 4

Use this checklist in the weeks before P4 begins. By launch day, every item should be checked off.

Week 1–2 Before Competition: Foundation

- ☐ Study all P1, P2, P3 products and strategies (this guide + listed repos)
- ☐ Install jmerle's backtester: `pip install prosperity3bt`
- ☐ Install jmerle's visualizer (browser-based, no install needed — see jmerle.github.io/imc-prosperity-3-visualizer/)
- ☐ Download all historical data from the backtester (rounds 0-7 are bundled)
- ☐ Run a 'hello world' trader to verify your environment works
- ☐ Implement and test market-making for a stable-price asset (Resin/Amethysts pattern)
- ☐ Implement and test Wall Mid market-making for a random-walk asset (Kelp pattern)

Week 3–4 Before Competition: Strategy Library

- ☐ Implement ETF basket z-score stat-arb template (from this guide)
- ☐ Implement BStoolkit (`call_price`, `implied_vol`, `delta`, `vega`)
- ☐ Implement BFS currency arbitrage solver
- ☐ Implement Monte Carlo fish auction simulator
- ☐ Implement Nash EV solver for container/suitcase games
- ☐ Build parameter injection into your Trader class for grid-search
- ☐ Run grid searches on P3 data to get familiar with the process

Competition Week: Round-by-Round

Round Opens	First 2 Hours	Hours 3-48	Hours 48-72
Tutorial/R1	Identify product archetypes. Plot all prices. Deploy market-maker.	Tune quote distances via backtest. Fine-tune inventory management.	Final submission. Do NOT change working code.
Round 2	Check for basket relationship. Compute synthetic fair values.	Implement z-score arb. Grid-search entry threshold. Test on all 3 days.	Final submission. Ensure stable PnL curve in backtest.
Round 3	Check for options. Compute IVs. Plot smile.	Fit smile. Implement delta hedge. Cap position at 80.	Final submission. Verify straight-line PnL in backtest.
Round 4	Look for conversion mechanism. Plot external signals vs price.	Find sunlight/environmental threshold. Test import/export basis.	Submit best available. Disable risky products if uncertain.
Round 5	Download de-anonymized trade data immediately.	Identify Olivia or equivalent. Build copy-trading signal. Grid search.	Final submission with all signals integrated.

The Ultimate Survival Rules

Rule 1: Never submit an untested algorithm. Always backtest first.

Rule 2: If you can't beat 0 PnL in backtest, set that product's orders to empty list.

Rule 3: Test your algorithm on EACH DAY separately. A strategy that works on Day 1 but fails on Day 2 will fail in production.

Rule 4: Implement inventory management from Day 1. An algorithm without it will hit position limits and stop trading.

Rule 5: Stable algorithm > perfectly optimized algorithm. Protect your existing PnL. Don't break working code in the final hours.

Rule 6: Read the entire competition wiki before each round. The problem description contains embedded hints about fair values and mechanics.

Rule 7: Compare new round data to all historical Prosperity data (P1, P2, P3). Cross-year data leaks happen. This is your free advantage.

Rule 8: The Discord is your most valuable real-time resource. Be active, be helpful, and you'll receive more hints than you give.

Rule 9: Simple works. Every top-3 team's core edge was simple, well-executed market-making and stat-arb — not complex ML models.

Rule 10: Study your losses. Download and visualize every log. The most valuable feedback is understanding why you lost money.

Sources Consulted in This Guide

Source	Team/Author	Competition	Placement
github.com/ShubhamAnandJain/IMC-Prosperity-2023-Stanford-Cardinal	Stanford Cardinal	P1	2nd Global
github.com/ericcccsliu/imc-prosperity-2	Linear Utility	P2	2nd Global
github.com/jmerle/imc-prosperity-2	camel_case (jmerle)	P2	9th Global
github.com/TimoDiehm/imc-prosperity-3	Frankfurt Hedgehogs	P3	2nd Global
github.com/chrispyroberts/imc-prosperity-3	CMU Physics	P3	7th Global, 1st USA
github.com/CarterT27/imc-prosperity-3	Alpha Animals	P3	9th Global, 2nd USA
github.com/MartinOravecSvK/IMC_Proprosperity_2025	Martin Oravec	P3	73rd Global, 5th UK
github.com/jmerle/imc-prosperity-3	camel_case (jmerle)	P3	25th Global
github.com/MichalOkon/imc_prosperity	Zahcheesha	P1	57th Global
github.com/gabsens/IMC-Prosperity-2-Manual	gabsens	P2	30th Manual
github.com/jmerle/imc-prosperity-3-backtester	jmerle (open source)	P3	N/A (tool)
dteather.com/blogs/imc-prosperity-2/	David Teather	P2	381st Global
medium.com/@oravec.martin01/imc-prosperity-3-be859180f133	Martin Oravec	P3	73rd Global

Good luck in IMC Prosperity 4. The market rewards preparation.